Katedra počítačov a informatiky

Fakulta elektrotechniky a informatiky

Technická univerzita v Košiciach

3D reprezentácia prevodovky

Počítačová grafika

Systémová príručka

Šk. rok: 2025/2026

Meno vyučujúceho: Ing. Timotej Sobota

Autori: Jakub Filičko,

Jana Kocáková

Obsah

Zoznam obrázkov 3
1. Funkcia program 4
2. Popis programu 4
2.1 Popis riešenia 4
2.1.1 Teoretický základ 5
2.2 Popis algoritmov a údajových štruktúr, globálnych premenných 6
2.2.1 Prehľad architektúry systému a workflow programu 6
2.3 Popis súborov, modulov, tried a podprogramov 7
2.3.1 Skript index.html 7
2.3.2 Skript main.js 7
3. Preklad programu 13
3.1 Požiadavky na technické prostriedky pri preklade 13
4. Nadväznosť na iné programové produkty 14
5. Zhodnotenie riešenia 14
6. Zoznam použitej literatúry 15

Zoznam obrázkov

Obrázok 1 Výpočet rýchlosti rotácie párov ozubených kolies 5
Obrázok 2 Lineárna interpolácia pre animáciu plynulého pohybu konektorov 6

1. Funkcia program

Tento program predstavuje **3D reprezentáciu prevodovky** automobilu vytvorenú v rámci tematického okruhu *Simulátor dynamického pohybu v 3D priestore*. Aplikácia umožňuje používateľovi dynamicky meniť atribúty a tým ovplyvňovať animovaný pohyb.

Hlavným cieľom projektu je priblížiť princíp činnosti manuálnej prevodovky a vizuálne znázorniť, ako sa mení prevodový stupeň a ako sa prenáša točivý moment z motora cez prevodovku až na kolesá. Používateľ môže sledovať celú poháňaciu sústavu (drivetrain), pričom dôraz je kladený najmä na samotnú prevodovku a jej vnútorné mechanizmy. Program slúži ako vzdelávací a interaktívny nástroj, ktorý zrozumiteľnou formou ukazuje spoluprácu jednotlivých komponentov. Výhodou riešenia je interaktivita – používateľ môže meniť uhol pohľadu, približovanie scény či otáčky motora, pričom simulácia rešpektuje princípy automobilovej techniky, aj keď niektoré prvky sú pre prehľadnosť mierne zjednodušené.

Používateľské rozhranie aplikácie je navrhnuté tak, aby bolo intuitívne a zrozumiteľné. V ľavej časti obrazovky sa nachádza okno s ovládaním a zoznamom klávesových skratiek, ktoré slúžia na pohyb kamery a zmenu prevodových stupňov. Na pravej strane sa nachádza ovládací panel, v ktorom si používateľ môže zvoliť režim kamery (voľný pohyb alebo preddefinované animované zábery), upraviť pokročilé nastavenia osvetlenia, zobrazenie karosérie vozidla a rýchlosť otáčania motora. Súčasťou aplikácie je aj sekcia *About*, ktorá obsahuje informácie o autoroch projektu, akademickom roku a príslušnosti k univerzite.

2. Popis programu

2.1 Popis riešenia

Pri návrhu a implementácii systému bolo potrebné zabezpečiť realistické správanie ozubených kolies, najmä z hľadiska vzájomnej väzby otáčania medzi hnacím a hnaným kolesom. Osobitný dôraz bol kladený na:

- správnu geometriu ozubených kolies,
- zachovanie prevodového pomeru,
- konzistentnú rýchlosť a smer rotácie medzi prepojenými kolesami,
- možnosť zmeny parametrov bez narušenia animácie.

Pre potreby návrhu prevodového systému, ktorý zodpovedá reálnym prevodovým systémom, sme použili znalosti z nasledujúcich zdrojov.

Najužitočnejší zdroj pre pochopenie preradzovania jednotlivých stupňov bola animácia 4-prevodového systému od learnengineering.org na platforme youtube. Pri takomto komplexnom systéme bola táto animácia najužitočnejšia pre pochopenie funkcií jednotlivých komponentov a pohybu komponentov pri zmene prevodu.

2.1.1 Teoretický základ

2.1.1.1 Prevodové mechanizmy

Základom simulácie je princíp ozubeného prevodu, podľa ktorého je pomer uhlových rýchlostí dvoch ozubených kolies **nepriamo úmerný** pomeru ich počtov zubov:

$$\frac{\omega_1}{\omega_2} = \frac{z_2}{z_1}$$

kde:

- ω_1, ω_2— uhlové rýchlosti kolies
- z_1, z_2— počty zubov kolies 1

V implementácii simulácie sa tento vzťah prenáša do výpočtu otáčkovej rýchlosti:

```
// Calculate speeds for each pair
const speed12 = baseSpeed * (T.c1 / T.o2);
const speed34 = baseSpeed * (T.c3 / T.o4);
const speed56 = baseSpeed * (T.c5 / T.o6);
const speed78 = baseSpeed * (T.i7 / T.c8);
```

Obrázok 1 Výpočet rýchlosti rotácie párov ozubených kolies

2.1.1.2 Lineárna interpolácia (LERP)

Na zabezpečenie plynulej animácie pohybu radiacich prvkov (konektory, radiaca páka) sa používa **lineárna interpolácia**:

$$x_{\text{new}} = x_{\text{current}} + (x_{\text{target}} - x_{\text{current}}) \cdot \alpha$$

kde:

- $\alpha \in (0,1)$— interpolačný faktor

```
// Smoothly interpolate only x position
const lerpSpeed = 0.1;
connector1.position.x += (targetX1 - connector1.position.x) * lerpSpeed;
connector2.position.x += (targetX2 - connector2.position.x) * lerpSpeed;
```

Obrázok 2 Lineárna interpolácia pre animáciu plynulého pohybu konektorov

2.2 Popis algoritmov a údajových štruktúr, globálnych premenných

2.2.1 Prehľad architektúry systému a workflow programu

Program sa inicializuje postupným zavolaním kľúčových funkcií: *init()* najprv vytvára scénu, renderer a základné nastavenia, potom *setupCamera()* inicializuje hlavnú kameru a animačné kamery, *setupLighting()* pripravuje osvetlenie, *setupSceneObjects()* vytvára statické 3D objekty prostredia (cesta, obloha, automobil), l*oadGears()* načítava GLTF modely prevodovky a ozubených
kolies, *setupKeyboardControls()* a *setupGearshiftGUI()* pripravujú používateľské rozhranie a ovládanie, a nakoniec sa spúšťa hlavná animačná slučka *animate().*

Tok dát v systéme má hierarchickú štruktúru: centrálna Three.js scéna obsahuje všetky 3D objekty vrátane modelov prevodovky (ozubené kolesá, hriadele, radiacej páky, spojovacích elementov), objektov prostredia (automobil, cesta, skybox) a svetelných zdrojov. V každom snímku animačnej slučky sa aktualizuje stav systému - funkcia *updateGearRotation()* vypočíta rotácie ozubených kolies na základe aktuálne zaradeného prevodového stupňa a rýchlosti motora, pričom prepočítava prevody medzi jednotlivými hriadeľmi (vstupný → prevodový → výstupný → kolesá auta). Následne *updateGearSystem()* upravuje pozície konektorov a radiacej páky podľa stavu prevodovky, a *updateCamera()* spracováva pohyb kamery podľa vstupu z klávesnice. Všetky zmeny objektov v scéne sú nakoniec vykreslené funkciou *renderer.render()* na obrazovku.

Používateľský vstup riadi celý systém prostredníctvom klávesnice a myši: klávesnica (WASD, šípky, Shift) ovláda pohyb kamery v priestore, stlačenie klávesy G otvorí GUI radiaceho mechanizmu, kde myš slúži na výber prevodového stupňa cez H-pattern rozhranie. Tento výber zmení globálnu premennú *currentGear*, čo spustí prestavenie celej prevodovky - posun konektorov, zmenu polohy radiacej páky podľa H-pattern schémy a prepočet rotácií všetkých ozubených kolies. Nastavenia scény (režim kamery, osvetlenie, viditeľnosť objektov) sa ovládajú cez *dat.GUI* panel v pravom hornom rohu,

ktorý umožňuje dynamické prepínanie medzi hlavnou kamerou, animačnými kamerami a upravovanie parametrov osvetlenia a viditeľnosti objektov v reálnom čase.

2.2.1.1 Data flow diagram

```
Užívateľ
  ├── WASD/↑ ↓ → keys → handleCameraMovement() → camera.position
  ├── G + myš → selectedGear → currentGear → updateGearSystem()
  └── dat.GUI → guiSettings → updateLighting(), animate()
                      ↓
                updateGearRotation()
                      ↓
            ozubené kolesá (rotation.x)
                      ↓
            outputshaft, tire rotation
                      ↓
             renderer.render(scene)
```

2.3 Popis súborov, modulov, tried a podprogramov

2.3.1 Skript index.html

Účel: Bootstrap aplikácie; vkladá canvas, modálne okno "About", H-pattern UI pre radenie.

Hlavné zodpovednosti: Pripraví DOM prvky (#canvas1, #about-modal, #gearshift-gui) a načíta knižnice a main.js.

Závislosti: Three.js, OrbitControls, GLTFLoader, dat.GUI, GSAP.

Verejné funkcie: Žiadne (HTML host, poskytuje UI prvky a skriptové zdroje).

2.3.2 Skript main.js

Účel: Hlavná logika 3D scény prevodovky, kamera, osvetlenie, načítanie modelu, animácie, ovládanie a vykresľovanie.

Hlavné zodpovednosti: Inicializácia scény a rendereru, nastavenie kamier a svetiel, načítanie GLTF prevodovky, riadenie rotácií kolies a hriadeľov podľa zaradeného stupňa, správa UI (About, H-pattern), slučka vykresľovania.

Závislosti: Three.js (core), OrbitControls, GLTFLoader, dat.GUI, GSAP; DOM prvky z *index.html*.

Verejné (globálne dostupné) funkcie*: init(), setupCamera(), setupLighting(), setupSceneObjects(), loadGears(), setupKeyboardControls(), setupAboutModal(), setupGearshiftGUI(), setupSettingsMenu(), setupCameraAnimation(), setupVerticalCameraAnimation(), updateGearRotation(), updateGearSystem(), updateLighting(), handleCameraMovement(), updateCamera(), onResize(), setupLightingGUI(), setupCarGUI(), updateRoofVisibility(), updateCarBodyVisibility(), animate(), pomocné: createRoadPlane(), createShaftLines(), openGearshift(), closeGearshift(), updateGearIndicator().*

2.3.2.1 Hierarchia scény podľa stromu z .glb exportu

Objekty, ktoré načítavame z .glb exportu majú nasledujúcu hierarchiu / strom, ktorá zodpovedá stromu objektov v Blendri. Vzťah parent-child exportovaná z Blendru do .glb formátu je výhodná pre priradzovanie pohybu / rotácie objektom, ktoré majú vzťah parent – child (napr. disk kolesa je childom pneumatiky kolesa).

glb.scene:

```
├── car_root (car)
│   ├── top001 (roofLeft)
│   ├── top (roofRight)
│   ├── tire_front_left
│   ├── tire_front_right
│   ├── tire_back_left
│   ├── tire_back_right
│   ├── drivetrain_shaft
│   └── shaft_to_wheels
├── cgear1
├── ogear2
├── cgear3
├── ogear4
├── cgear5
├── ogear6
├── igear7
│   └── inputshaft          (pripojený v kóde ako child igear7)
├── cgear8
│   └── countershaft        (pripojený v kóde ako child cgear8)
├── connector1
│   └── right_shiftfork     (pripojený v kóde ako child connector1)
├── connector2
│   └── left_shiftfork      (pripojený v kóde ako child connector2)
├── outputshaft
└── gearlever
```

2.3.2.2 Fyzické prepojenie komponentov prevodovky

connector1 ⟷ right_shiftfork ⟷ cgear1/cgear3
(posúva sa v osi X) (na countershaft)

connector2 ⟷ left_shiftfork ⟷ cgear5/igear7
(posúva sa v osi X) (na outputshaft/inputshaft)

gearlever → vizuálna indikácia (rotácia X,Z podľa H-pattern)

2.3.2.3 Popis kľúčových funkcií

init(): Vstupy: -, Výstupy: -

Algoritmus: Vytvorí scene, renderer, nastaví enkódovanie, tieňovanie, vloží canvas do #canvas1, zaregistruje onResize.

Globálne: scene, renderer.

setupCamera():

Vstupy: -, Výstupy: -

Algoritmus: Inicializuje hlavnú kameru, OrbitControls s targetom pred kamerou; vytvorí animationCamera (horizontálny okruh) a verticalAnimationCamera, spustí ich GSAP trajektórie.

Globálne: camera, controls, animationCamera, verticalAnimationCamera.

setupCameraAnimation(), setupVerticalCameraAnimation():

Vstupy: -, Výstupy: -

Algoritmus: Definuje cyklické GSAP timeline pre pozície kamier; pri update nastavuje lookAt na stred prevodovky.

Globálne: animationCamera alebo verticalAnimationCamera.

setupLighting():

Vstupy: -, Výstupy: -

Algoritmus: Pridá AmbientLight, konfiguruje DirectionalLight so shadow parametrami, vytvorí SpotLight (headlight).

Globálne: ambientLight, directionalLight, headlight, scene.

updateLighting():

Vstupy: -, Výstupy: -

Algoritmus: Podľa aktívnej kamery posúva a smeruje headlight do smeru pohľadu, aktualizuje target.

Globálne: headlight, camera, animationCamera, verticalAnimationCamera, guiSettings.

setupSceneObjects(), createRoadPlane(texLoader):

Vstupy: -, Výstupy: -

Algoritmus: Načíta skybox textúru do veľkej sféry; vytvorí cestu (plane) s opakovaním textúry a materiálom; nastaví pozíciu, príjem tieňov.

Globálne: skyboxSphere, roadPlane, roadTexture, scene.

loadGears():

Vstupy: -, Výstupy: -

Algoritmus: Pomocou GLTFLoader načíta gearbox16.glb, nastaví tieňovanie, vyhľadá objekty (kolieska, hriadele, konektory, vidlice, pneumatiky, karosériu, radiacu páku), uloží neutrálne stavy, presunie a zreťazí komponenty do správnych rodičov s korekciou pozícií, vytvorí pomocné "shaft lines".

Globálne: množstvo objektov scény (cgear1..., connector1/2, inputshaft, countershaft, outputshaft, car, gearlever, atď.), neutrálne pozície/rotácie.

createShaftLines(shaft1, shaft2, shaft3):

Vstupy: shaft1, shaft2, shaft3 (Mesh)

Výstupy: -

Algoritmus: Pridá cylindrické "čiary" na hriadele pre vizuálnu indikáciu priemerov.

Globálne: -

updateGearRotation():

Vstupy: -, Výstupy: -

Algoritmus: Vypočíta rýchlosti párov podľa zubov (T), základnú rýchlosť z guiSettings.car.engineSpeed, aplikuje rotácie kolies a hriadeľov (vzájomné prevody).

Globálne: Ozubené kolesá cgear*, ogear*, igear7, cgear8, guiSettings.

updateGearSystem():

Vstupy: -, Výstupy: -

Algoritmus: Podľa currentGear lineárne posúva connector1/2, vypočíta a aplikuje rotáciu outputshaft, "posúva" cestu (roadTexture.offset), riadi rotáciu pneumatík; riadenie radiacej páky prebieha v stavoch (gearLeverMovementState) s "H-pattern" prechodom cez neutrál pri zmene strán.

Globálne: currentGear, previousGear, connector1/2, gearlever, neutrálne/target rotácie páky, roadTexture, pneumatiky, outputshaft, guiSettings.

setupKeyboardControls():

Vstupy: Eventy keydown/keyup.

Výstupy: -

Algoritmus: Nastavuje booleany v keys pre WASD/ ↑ / ↓ /Shift; G otvára/zatvára H-pattern GUI.

Globálne: keys, isGearshiftOpen, volá openGearshift()/closeGearshift().

setupAboutModal():

Vstupy: Eventy kliknutia.

Výstupy: -

Algoritmus: Prepína CSS triedy na zobrazenie/skrytie modálu "About".

Globálne: DOM prvky modálu.

setupGearshiftGUI(), openGearshift(), closeGearshift(), updateGearIndicator():

Vstupy: mousemove pri otvorenom GUI, kliknutia; dotyk bodu v H-pattern.

Výstupy: -

Algoritmus: Počíta najbližšiu pozíciu v H-pattern (ľavý/pravý vertikál, stredný horizontál), "snapuje" bod, nastavuje selectedGear a zvýraznenie; otvorenie/uzavretie GUI mení isGearshiftOpen a controls.enabled, po zavretí prekopíruje selectedGear do currentGear.

Globálne: selectedGear, currentGear, isGearshiftOpen, controls.

setupSettingsMenu(), setupCameraGUI(), setupLightingGUI(), setupCarGUI():

Vstupy: -, Výstupy: -

Algoritmus: Vytvorí dat.GUI, sekcie pre kameru, osvetlenie a auto; binduje ovládacie prvky na viditeľnosť, FOV, režimy kamery, parametre svetla, animácie karosérie, viditeľnosť cesty/strechy.

Globálne: gui, guiSettings, camera, animationCamera, verticalAnimationCamera, ambientLight, directionalLight, headlight, car, roadPlane, roofLeft/Right.

handleCameraMovement():

Vstupy: -, Výstupy: boolean (či sa kamera pohla).

Algoritmus: Z keys skladá pohybový vektor v lokálnych osiach (forward/right/up), upraví camera.position.

Globálne: camera, keys, rýchlosti moveSpeed/fastMoveSpeed.

updateCamera():

Vstupy: -, Výstupy: -

Algoritmus: Volá handleCameraMovement(), ak sa kamera pohla, aktualizuje controls.target do smeru pohľadu a controls.update().

Globálne: camera, controls, keys.

onResize():

Vstupy: -, Výstupy: -

Algoritmus: Aktualizuje aspect a projekčné matice aktívnych kamier.

Globálne: camera, animationCamera, verticalAnimationCamera.

updateRoofVisibility(visible), updateCarBodyVisibility(visible):

Vstupy: visible: boolean, Výstupy: -

Algoritmus: Prepína viditeľnosť strechy; animuje karosériu GSAPom (z oblohy na pôvodnú pozíciu a späť).

Globálne: roofLeft/Right, car, carOriginalPosition.

animate():

Vstupy: -, Výstupy: -

Algoritmus: Slučka requestAnimationFrame; volá updateGearRotation(), podmienene updateGearSystem(), podľa režimu kamery updateCamera() alebo lookAt, vždy updateLighting(), vyberie aktívnu kameru a renderer.render(scene, activeCamera).

Globálne: renderer, scene, kamery, guiSettings.

2.3.2.4 Hierarchia volaní po štarte

```
Štart → init()
    → setupCamera() → setupCameraAnimation(), setupVerticalCameraAnimation()
    → setupLighting()
    → setupSceneObjects() → createRoadPlane()
    → setupSettingsMenu() → setupCameraGUI(), setupLightingGUI(), setupCarGUI()
    → loadGears() → createShaftLines()
    → setupKeyboardControls(), setupAboutModal(), setupGearshiftGUI()
    → animate() (nekonečná slučka)
        ├── updateGearRotation()
        ├── [ak načítané] updateGearSystem()
        ├── [režim 'main'] updateCamera() → handleCameraMovement()
        ├── updateLighting()
        └── renderer.render(scene, activeCamera)
```

Eventy: onResize() upravuje kamery; G otvára/zatvára H-pattern; dat.GUI mení nastavenia.

3. Preklad programu

Aplikáciu je možné spúšťať prostredníctvom prostredia WebStorm IDE, ktoré bolo štandardne používané v rámci predmetu *Počítačová grafika*. Používateľ otvorí súbor *index.html* a následne klikne na ikonu zvoleného webového prehliadača. Aplikácia sa spustí v prehliadači ako localhost.

3.1 Požiadavky na technické prostriedky pri preklade

Presné technické prostriedky nutné k hladkému priebehu animácie a prekladu je ťažké odhadnúť, ale prikladáme špecifikácie našich notebookov, na ktorých preklad a animácie prebiehali hladko a poskytovali výborný výkon aj napriek mnohým objektom v scéne a mnohým načítaným textúram.

Špecifikácia notebooku 1:
OS: Windows 11
Procesor: : 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz

Grafická karta: Intel Iris Xe, 512 MB
Operačná pamäť: 16 GB RAM

Špecifikácia notebooku 2:
OS: macOS
Procesor: MacBook Air Apple M1
Grafická karta:
Operačná pamäť: 16GB RAM

4. Nadväznosť na iné programové produkty

Pre tvorbu helikálnych ozubených kolies bola použitá webstránka *Otvinta*, ktorá má nástroj pre vykreslenie bázy ozubeného kolesa s možnosťou výpočtu uhla pre vytvorenie helikálnych prevodov.

Blender assety použité v tomto projekte sú vytvorené autormi, alebo stiahnuté z databázy Sketchfab. Všetky komponenty prevodovky boli vytvorené autormi v Blenderi. Komplexnejšie assety, ktoré pochádzajú z Sketchfab sú objektami motora, karosérie, radiacej páky, diferenciálu a kolies. Odkazy na použité assety sa nachádzajú v zozname použitej literatúry.

5. Zhodnotenie riešenia

Aktuálne riešenie poskytuje funkčnú vizualizáciu základného princípu prevodovky, avšak obsahuje niekoľko zjednodušení, ktoré by mohli byť predmetom ďalšieho vývoja. Najvýraznejšou technickou simplifikáciou je použitie jednoduchých konektorov (*connector1, connector2*) namiesto realistických komponentov ako synchronizátor a zubová spojka (dog clutch). V skutočnej prevodovke synchronizátor vyrovnáva otáčky ozubených kolies pred ich zopnutím a zabezpečuje plynulé radenie, zatiaľ čo zubová spojka fyzicky prenáša moment medzi hriadeľom a kolieskom. Implementácia týchto komponentov by vyžadovala rozšírenie 3D modelu v Blenderi o detailné geometrie synchronizačných krúžkov, zubových spojok a ich animované pohyby, ako aj prepracovanie logiky v *updateGearSystem()* o fázy synchronizácie (kontrola rozdielov otáčok, vyrovnanie rýchlostí, následné zopnutie). Ďalším obmedzením je fixný počet prevodových stupňov (4 + neutrál) bez spiatočky a absencia simulácie spojky, čo by bolo potrebné pre komplexnejšiu interakciu s používateľom.

Ďalší vývoj by mohol zahŕňať: (1) rozšírenie modelu o 5. a 6. stupeň, spiatočku a (2) simuláciu spojkového pedála cez klávesnicu alebo gamepad s vizualizáciou disku spojky a jeho zopnutia, (3) podporu VR zariadení pre imerzívny pohľad z vodičskej perspektívy s rukami ovládajúcimi radiacu páku, (4) analytické nástroje zobrazujúce real-time grafy prevodových pomerov, krútiaceho momentu a výkonu, (5) interaktívny tutoriál

vysvetľujúci mechanizmus prevodovky s možnosťou "rozklikať" komponenty a zobraziť ich funkciu.

6. Zoznam použitej literatúry

[1] Shigley, J. E. (2011). *Shigley's Mechanical Engineering Design*. McGraw-Hill.
[2] Lengyel, E. (2011). *Mathematics for 3D Game Programming and Computer Graphics*. Course Technology Press.
[3] Three.js Documentation: Camera (https://threejs.org/docs/#api/en/cameras/Camera)
[4] Computational Geometry: Distance Calculations (https://en.wikipedia.org/wiki/Euclidean_distance)
[5] Three.js Texture Mapping (https://threejs.org/docs/#api/en/textures/Texture)
[6] Phong, B. T. (1975). "Illumination for computer generated pictures". *Communications of ACM*.
[7] https://www.otvinta.com/instantgear.html
[8]https://www.youtube.com/watch?v=wCu9W9xNwtI&list=PL1x4vuU4RipBbw9GjSDR9ITATyplqs18b&index=13
[9] https://sketchfab.com/3d-models/classic-wheel-64c242c610dd4e8088f5f69394409c9d
[10] https://sketchfab.com/3d-models/1929-guilaume-busson-coupe-marquez-00d4df97a6454479ba4313c33b0e06bb
[11] https://sketchfab.com/3d-models/car-engine-2ee0cab6e1654f0eb5f92adc51ff4244#download
[12] https://sketchfab.com/3d-models/manual-transmission-gear-box-811ce409dda746099924ac0b39d34140